Project: Mini Bank Management System API

1. Project Overview

Develop a Mini Bank Management System using Django REST Framework (DRF). The system will allow customers to register, manage their bank accounts, deposit and withdraw money, transfer money to other accounts, and view their transaction history.

The primary objective of this assignment is to demonstrate your understanding of REST API development, database relationships, serializers, authentication, permissions, and basic business logic.

2. Functional Requirements

1: Customer Authentication

- Implement customer registration and login.
- Allow authenticated customers to view and update their profiles.
- Protect private endpoints so that only authenticated users can access them.
- Customers must not be able to access another customer's private information.

2: Bank Account Management

- Customers can create a bank account.
- Each account must have a unique account number.
- Each account must have an account type (Savings or Current).
- Customers can view their own accounts and account details.
- Customers can check their current account balance.

3: Deposit and Withdrawal

- Customers can deposit money into their accounts.
- Customers can withdraw money from their accounts.
- The system must reject zero or negative amounts.
- The system must prevent withdrawals when the available balance is insufficient.
- Each successful deposit or withdrawal must be recorded as a transaction.

4: Money Transfer

- Customers can transfer money from their account to another account.
- The system must verify that the sender has sufficient funds.
- Customers cannot transfer money to the same account.
- The sender's balance must decrease, and the receiver's balance must increase.
- Each successful transfer must be recorded in the transaction history.

5: Transaction History

- Customers can view the transaction history of their own accounts.
- Each transaction must include the transaction type, amount, date, and relevant account information.
- Customers can filter transactions by transaction type.
- Transaction history must be displayed in a paginated format.

6: Admin Management

- Admin users can view all customers and bank accounts.
- Admin users can view transaction records.
- Only authorized admin users can perform administrative operations.

3. Technical Requirements

Your project must meet the following technical requirements:

1. Use Django REST Framework to build the APIs.
2. Use Django models to define the database structure.
3. Use serializers for data conversion and validation.
4. Implement appropriate authentication and permission classes.
5. Use proper relationships between customers, accounts, and transactions.
6. Return appropriate HTTP status codes and meaningful error messages.
7. Use `DecimalField` for monetary values.
8. Follow a clean project structure and maintain readable code.
9. Test all major API endpoints using Postman.
10. Provide a README file explaining the project setup and how to use the APIs.

4. Suggested API Endpoints

You may organize your endpoints differently, provided that all required features are implemented.

HTTP Method	Endpoint	Purpose
POST	`/api/auth/register/`	Register a customer
POST	`/api/auth/login/`	Customer login

GET	`/api/profile/`	View own profile
PATCH	`/api/profile/`	Update own profile
GET	`/api/accounts/`	List own accounts
POST	`/api/accounts/`	Create an account
GET	`/api/accounts/<id>/`	View account details
GET	`/api/accounts/<id>/balance/`	Check account balance
POST	`/api/transactions/deposit/`	Deposit money
POST	`/api/transactions/withdraw/`	Withdraw money
POST	`/api/transactions/transfer/`	Transfer money
GET	`/api/transactions/`	View transaction history

5. Database Models

Your project should include at least the following models:

- **User/Customer:** Customer information and authentication.
- **BankAccount:** Account number, account type, balance, owner, and status.
- **Transaction:** Transaction type, amount, related account or accounts, timestamp, and transaction reference.

You may add additional fields or models if required.

6. Business Rules

Your system must follow these rules:

- Every account number must be unique.
- Account balances must never become negative.
- Deposits, withdrawals, and transfers must use valid positive amounts.
- Only the account owner can perform transactions on their own account.
- Customers must not access other customers' private accounts or transaction histories.
- Failed transactions must not change account balances.
- A successful transfer must update both account balances consistently and create the relevant transaction records.

7. Submission Guidelines

Every student must submit the following:

1. **GitHub Repository URL:** Submit the public GitHub repository link containing your complete project source code.
2. **README.md:** Include project description, installation instructions, environment setup, database migration commands, and instructions for running the server.

9. Bonus Challenge (Optional)

Students who want to earn additional credit may implement one or more of the following:

- Filter transactions by date range.
- Add daily withdrawal limits.
- Write unit tests for deposit, withdrawal, and transfer operations.
- Generate a downloadable account statement.

Final Instructions

Focus on correctness, security, clean code, and proper API behavior. A working API with correct validation and permissions is more important than building a frontend.

Good luck, everyone!